

Lab 2: Software Configuration Management and Project Tracking

Supporting tools: Git and GitHub

Lab objective(s)

Understanding of:

- Why change control and work tracking are engineering activities in a software project.
- Configuration management in practice: repositories, versions, branches and baselines.
- Controlling change through pull requests and code review, and resolving merge conflicts.
- Planning and tracking work with Issues, Labels, Milestones and Projects.
- Assigning work, estimating effort, and reading a project schedule as a table and a timeline.
- Traceability from a deliverable to the change that produced it.

Course learning outcomes: **CLO 1** (iterative and incremental development), **CLO 5** (professional practice: honest attribution of work), **CLO 6** (working in a group to design, implement, test and present a software system).

Where this fits in software engineering

Software is built by teams, over time, under continuous change. Two problems appear as soon as more than one engineer works on the same system. **Change must be controlled:** every change recorded, attributable and reversible, and nothing entering the released version without review. That is **software configuration management**. **Work must be planned and tracked:** broken down, assigned, prioritised, scheduled against deliverables, and its progress visible to the team. That is **software project management**.

These are engineering activities, not tools. A tool is then chosen to support them; here it is **Git**, a version control system, with **GitHub**, which hosts Git repositories and adds change control and work tracking on top of them.

Engineering activity	What it requires	Supported by
Version control and parallel development	every change recorded and reversible; people working at once without interference	Git commits, history and branches
Change control and baseline	nothing enters the known-good version unreviewed	pull requests, reviews, a protected <code>main</code>
Work breakdown and assignment	tasks small enough to own and to track	Issues, assignees, labels
Scheduling against deliverables	deadlines, and completion that can be measured	Milestones
Progress monitoring	the current state visible to the whole team	Projects (boards)
Traceability	a deliverable linked to the change that produced it	issue, branch, commit, pull request

Part A: Version control and change control

A.1 Git, GitHub, and repositories

Git is a *distributed* version control system: every copy of the project holds the full history, so you can commit and branch without a network connection. **GitHub** hosts Git repositories and adds the collaboration features used here.

A **repository** is a project folder that Git tracks, together with the complete history of every change. You use two, both private, created when you accept the assignment link posted on Blackboard.

Repository	Naming format	Example	Contents
Individual	section-username-firstname	b55-dd0123456-dane	your lab exercises
Team	team-sectionNo-teamNo	team-B05-G02	the project milestones

Keep a `README.md` at the root describing the project and its members, and a `.gitignore` listing what must never be tracked: build output, IDE settings, and anything holding a password or key ([ready-made templates](#)). Deleting a secret in a later commit does **not** remove it from the history.

The project produces documents, diagrams and code. Git versions all of them, but it can show a line-by-line difference only for text, such as source code and Markdown. For a `.docx` file or an exported diagram it records *that* the file changed, not *what* changed inside it, so the commit message is the only record: write it accordingly. Where a diagram tool can also save a text-based source file, commit that alongside the exported image.

A.2 Setting up

1. Create a GitHub account and verify your QU e-mail address. The username you choose appears on every commit you push.
2. Sign up for the [GitHub Student Developer Pack](#).
3. Install [Git](#), and at least one of [GitHub Desktop](#) or [Visual Studio Code](#). Sign in through the app so that it stores your credentials. GitHub no longer accepts an account password on the command line; there you would use a personal access token or an SSH key.
4. Tell Git who you are, or your commits will not be linked to your account:

```
git config --global user.name "Your Full Name"
git config --global user.email "your-verified-email@example.com"
```

A.3 Recording a change

Your work passes through four places:

working directory → **add** → staging area → **commit** → local repository → **push** → GitHub

Do the work in whichever application you prefer. The equivalent command is given in the last column: it is identical on every machine, and it is what the applications run for you.

Step	GitHub Desktop	VS Code	Command
Clone the repository onto your computer	File > Clone repository	Source Control > Clone Repository	<code>git clone <url></code>
See what has changed	the <i>Changes</i> tab	the <i>Source Control</i> panel	<code>git status</code>
Stage a change for the next commit	tick the file's checkbox	the + beside the file	<code>git add <file></code>
Commit it to the local repository	write a summary, <i>Commit to main</i>	type the message, then ✓ <i>Commit</i>	<code>git commit -m "..."</code>

Step	GitHub Desktop	VS Code	Command
Push the commits to GitHub	<i>Push origin</i>	<i>Sync Changes</i>	<code>git push</code>
Fetch without changing your files	<i>Fetch origin</i>	<i>... > Pull, Push > Fetch</i>	<code>git fetch</code>
Pull teammates' changes into your copy	<i>Fetch origin, then Pull origin</i>	<i>Sync Changes</i>	<code>git pull</code>

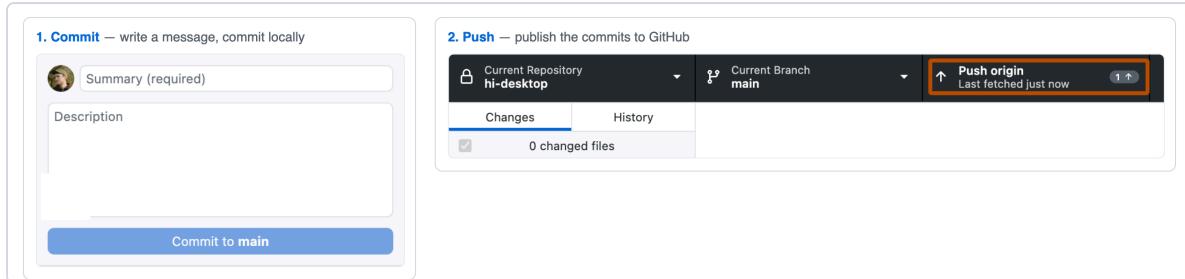


Figure 1. GitHub Desktop runs the same operations as the commands above. Left: the commit message fields and the *Commit to main* button. Right: the repository bar, showing the current repository, the current branch, and *Push origin*.

The everyday cycle is **pull** → **edit** → **add** → **commit** → **push**. Commit often, and write messages that say *why*: docs: add use-case diagram for Milestone-1, never update or final v3.

Do not clone a repository into a Google Drive, OneDrive or Dropbox synchronised folder. The sync client interferes with the hidden `.git` folder and the history can be corrupted.

A.4 Branches and pull requests

A **branch** is an independent line of development, so that `main` always holds a working version: the team's baseline. Create one branch per task, and delete it once merged.

Step	GitHub Desktop	VS Code	Command
Start from the latest <code>main</code>	<i>Current branch > main, then Pull origin</i>	the branch name in the status bar, then <i>Sync Changes</i>	<code>git switch main && git pull</code>
Create a branch	<i>Current branch > New branch</i>	<i>... > Branch > Create Branch</i>	<code>git switch -c feature/login-screen</code>
Publish it to GitHub	<i>Publish branch</i>	<i>Sync Changes</i>	<code>git push -u origin feature/login-screen</code>
Switch between branches	<i>Current branch > ...</i>	click the branch name in the status bar	<code>git switch <name></code>

A **pull request** proposes merging a branch into `main`, and is the point at which change is controlled: publish the branch, then open the pull request from GitHub Desktop (*Preview Pull Request*), from VS Code with the *GitHub Pull Requests* extension, or on github.com (*Compare & pull request*). Describe what changed and why, and request a reviewer. The reviewer comments on **Files changed** and approves; the pull request is then merged and the branch deleted. A repository can be set to require this and to refuse direct commits to `main` (*Settings > Branches*), which is how the baseline is protected.

A.5 Merge conflicts

A **conflict** occurs when two people change the same lines of the same file, so Git cannot decide which version to keep. It is a normal event, not an error. Git marks the file:

```
<<<<<<< HEAD
the version on your branch
=====
```

```
the version coming from main
>>>>>> main
```

In **VS Code** the conflicting file opens with *Accept Current Change*, *Accept Incoming Change* and *Accept Both Changes* offered above each conflict; **GitHub Desktop** lists the conflicted files and opens them in your editor. Whichever you use, the file must end up holding the agreed final text and none of the markers. Then stage, commit and push it as usual (`git add`, `git commit`, `git push`). The people who wrote the two versions decide together which text is correct.

Part B: Planning and tracking the project

B.1 Issues: the work breakdown

An **issue** is one unit of work: a task, a bug or a question. Open one from **Issues > New issue**. A useful issue has a title that names the work, a body containing the acceptance criteria as a checklist (- [] creates a tick box), an **assignee**, **labels** and a **milestone**.

Split the work until each issue is small enough for **one person to finish in one sitting**. Anything larger is not yet a task. Any issue can be referred to anywhere on GitHub by its number, **#12**.

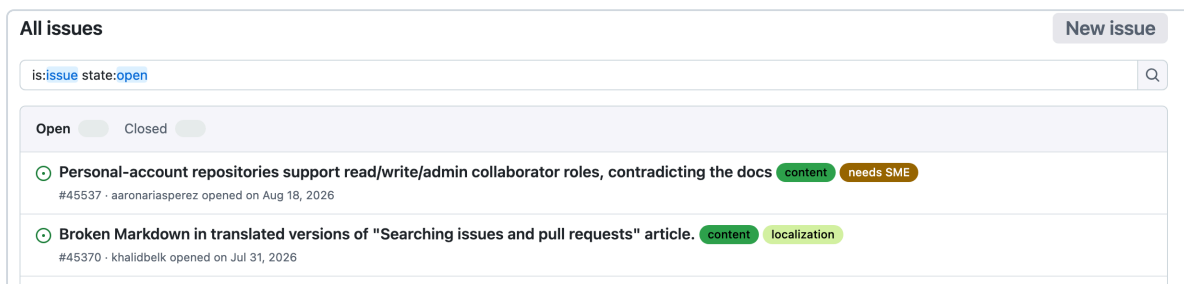


Figure 2. A list of issues. Each row is one unit of work, carrying its number, its author, and the labels that classify it.

B.2 Labels and milestones

Labels classify issues so that they can be filtered later (**Issues > Labels**). A small, consistent set is far more useful than many ad-hoc ones.

Label group	Purpose
type: feature / type: bug / type: docs	what kind of work it is
priority: high / medium / low	what to do first
phase: requirements / design / implementation / testing	which stage of the project it belongs to

A **milestone** groups the issues of one deliverable and gives them a **due date** (**Issues > Milestones**). Create one for each graded deliverable of the course project, **Milestone-1** and **Milestone-2**, using the due dates in the syllabus. The milestone page then shows the percentage complete and how many issues remain, a progress report nobody has to write.

B.3 Projects: monitoring progress

A **Project** (the **Projects** tab) is a board showing the state of every issue at a glance, with columns for its status: **Todo** → **In Progress** → **In Review** → **Done**. These are status values of a single item, not different kinds of task: the same issue moves across the board as it progresses.

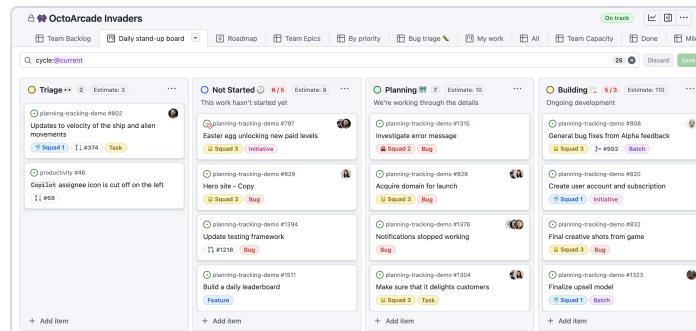


Figure 3. A project board. Each card is an issue, showing its number, assignee, labels and estimate; each column is a status. The counts and estimates in the column headers give the team its current load at a glance.

Under **Workflows**, built-in automation moves an item to *Done* when its issue is closed or its pull request is merged, so the board stays accurate without anyone maintaining it by hand.

B.4 Assigning work, estimating it, and seeing the schedule

Two questions a team has to answer every week: **who is doing what**, and **will it be finished in time**. A project answers both, provided its items carry the right fields.

Assign every issue to **one** person; work owned by "the team" is owned by nobody. Add the other fields to the project with the + at the right-hand end of the field row.

Field	Type	What it is for
Assignees	built in	the one person responsible for the item
Size or Estimate	single select, or number	the effort: <i>XS</i> / <i>S</i> / <i>M</i> / <i>L</i> , or hours
Start date	date	when work on the item begins
Target date	date	when it has to be finished
Iteration	iteration	the week, or sprint, that the item belongs to

The same items can then be read in three ways; switching view duplicates nothing.

Table view lists every item as a row with all of its fields: type, title, progress, status, assignee, linked pull request, priority. Group it by **Assignees** to check the workload, since each group shows its own item count and the total of its estimates. If one member's total is three times anyone else's, rebalance now rather than in the last week.

Type	Title	Sub-issues progress	Status	Assignees	Linked pull requests	Priority
Squad 1 (Estimate: 4)						
1 Task	Updates to velocity of the ship and alien mo...	#802 1/2 50%	Planning	azenMatt	#374	Urgent
2 Epic	Task Management	#39 2/6 33%	Review	rileybroughten		High
3 Bug	Search functionality returns no results for ex...	#1411 0/1 0%	Triage	rileybroughten	#1218	Low
Squad 2 (Estimate: 3)						
4 Task	Start explorations for game site	#801 1/2 50%	Building	ajashams and...	#993	High
5 Initiative	Social Media Campaign	#19 0/4 0%	Not Started	LisaKr		High
6 Epic	Integration and Automation	#42 0/5 0%	Not Started	azenMatt		Low
7 Epic	Reporting and Analytics	#41 0/8 0%	Planning	sunny101		Medium
Squad 3 (Estimate: 2)						

Figure 4. Table view, grouped into teams. Each row carries the assignee, the status, the linked pull request and the priority, and each group header shows its item count and the sum of its estimates.

Roadmap view places each item on a calendar using its start and target dates. This is the project's Gantt chart: the bars show how long each task runs and where tasks overlap, the avatars show who owns each one, and a

milestone appears as a marker on the timeline, so the team can see whether the work planned for Milestone-1 actually fits before its due date.

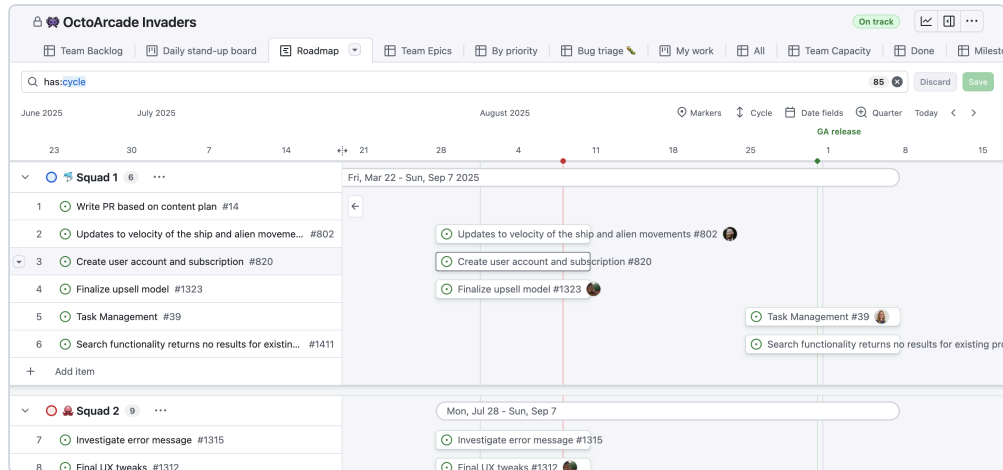


Figure 5. Roadmap view: the same issues placed on a timeline by their start and target dates, grouped by team, with a milestone marked on the calendar and a line showing today.

B.5 Traceability

Keeping the plan and the code in one place means any deliverable can be traced back to the change that produced it.

- From an issue, use **Create a branch** (right sidebar, under *Development*). GitHub names the branch after the issue and links the two.
- Refer to the issue in commit messages: `git commit -m "docs: draft problem statement (refs #7)"`.
- Write **Closes #7** in the pull request description. Merging it closes the issue automatically and the board moves the item to *Done*.

deliverable → issue #7 → branch 7-problem-statement → PR #12 → merge → Done

The repository's **Insights** tab (*Contributors*, *Network*) then shows who contributed what. *Figures 1, 3, 4 and 5: GitHub documentation, © GitHub, Inc., CC BY 4.0. Figure 2: a public GitHub repository.*

Part C: Reference

The commands behind the buttons used above, for working without a graphical client.

```
git clone <url>           # copy a remote repository locally
git status                # what has changed?
git switch main && git pull # start from the latest main
git switch -c feature/my-task # create and switch to a branch
git add <file>            # stage a change (git add . stages everything)
git commit -m "type: summary" # record it locally
git push -u origin feature/my-task # publish a new branch
git push                  # subsequent pushes
git merge main            # bring main's changes into your branch
git restore <file>        # discard uncommitted changes to a file
```

Git: [Pro Git \(the free official book\)](#) · [Set up Git](#) · [Git cheat sheet \(PDF\)](#) · [GUI clients](#)

Practice: [GitHub Skills](#) · [Learn Git Branching](#) · [CMPS 310 Version Control guide](#)

Branches and pull requests: [Branches](#) · [Pull requests](#) · [Reviews](#) · [Resolving a merge conflict](#)

Planning and tracking: [Issues](#) · [Labels](#) · [Milestones](#) · [Projects](#) · [Linking a pull request to an issue](#)

Writing and setup: [Markdown syntax](#) · [.gitignore templates](#) · [Conventional Commits](#) · [GitHub Desktop docs](#)